

Merge & Deploy Guide — Everything Needed to Hand This to Claude Code

This is the final handoff document. Everything below exists so a Claude Code session (in your GitHub Codespace) can merge all the delivered zips into one real repository, get it running, and prepare it for deployment — without having to re-derive any of this from scratch or guess at what's missing.

Step 1 — Gather every package, in this order

1. Your ORIGINAL base repo (the one with app/database.py, app/config.py, the Phase-1 bot evaluators, and the existing frontend App.tsx — everything this whole conversation has been layering additions onto).
2. petrazim_v2_phase1to6_additions.zip
3. petrazim_v3_phase1and2.zip
4. petrazim_v3_explore_concepts_1to5.zip
5. petrazim_v3_reference_aligned_lms.zip
6. petrazim_v3_onboarding_corporate_seats.zip
7. petrazim_v3_MASTER_consolidated.zip (rollup of the TradingView widget + curriculum content round)
8. petrazim_v3_facilitator_calendar_and_visual_updates.zip
9. petrazim_v3_access_expired_card.zip
10. petrazim_v3_weekly_cycle_and_email.zip
11. petrazim_v3_gradient_design_system.zip
12. The last few petrazim_preview_v*.jsx artifact files (the finished visual reference — these are single-file previews, not meant to be merged into the codebase directly, but Claude Code should use the LATEST one, v9, as the visual source of truth when reconciling the real component files against it).

Each numbered zip's own MERGE_MANIFEST.md (they all share and append to the same file) documents exactly which files are in it and any known conflicts – Claude Code should read that first.

Step 2 – The prompt to give Claude Code

```
I have an original Petrazim Trading Platform repo plus 10
zip files
that were built incrementally in a chat conversation with
Claude
(each zip's MERGE_MANIFEST.md documents what's in it and
known
conflicts between them). Unzip all of them into this repo,
in the
order listed in MERGE_MANIFEST.md, resolving conflicts per
that
document (in every case, the LATER package's version wins).
```

Then:

1. Wire every new router into app/main.py (auth, admin, payments, payment_webhooks, corporate, telegram_webhook, onboarding, monte_carlo, validation_gate, weekly_review, chart_layouts, facilitator).
2. Apply the App.tsx routing changes described in App.tsx.DIFF_NOTES.txt.
3. Run `pip install -r requirements.txt -r requirements-additions.txt` and `npm install`.
4. Run the SQL migrations in order (001 through 004) against a local or dev Postgres instance.
5. Start both servers locally and confirm they boot without import errors – this is the real integration test chat-based review couldn't do, since files were written across many separate sandbox

```
sessions without ever running together until now.
6. Fix any import/reference errors you find – they're
expected, given
    how this was built, and should be quick once you have
real file
    access and a real Python/Node error trace to work from.
7. Use petrazim_preview_v9.jsx as the visual reference for
the
    dashboard, nav, and card styling – reconcile the real
component
    files (TopNav.tsx, FoldedCard.tsx, the page components)
against it
    so the running app actually looks like that preview, not
just the
    individually-built components from earlier in the
conversation.
```

Step 3 – Environment variables, consolidated

Every one of these was flagged individually as a placeholder somewhere in the build. This is the complete list in one place – nothing here has a real value yet; these are the names the code expects.

Core (required for anything to run)

Variable	Used by
DATABASE_URL	Postgres connection, postgresql+asyncpg://...
SECRET_KEY	JWT signing – must not be the placeholder default

Payments

Variable	Used by
STRIPE_SECRET_KEY	Stripe checkout (test mode: sk_test_...)

Variable	Used by
STRIPE_WEBHOOK_SECRET	Verifying Stripe webhook payloads
PAYSTACK_SECRET_KEY	Paystack checkout (test mode: sk_test,...)
PAYSTACK_WEBHOOK_SECRET	Verifying Paystack webhook payloads

Telegram

Variable	Used by
TELEGRAM_BOT_TOKEN_INDIVIDUAL	@petrazim_tradefx_bot — regenerate this token via @BotFather first, the original was shared in chat
TELEGRAM_BOT_TOKEN_CORP	@petrazim_tradefx_corp_bot — same regeneration note applies

Facilitator meetings

Variable	Used by
FIREFLIES_API_KEY	Notetaker auto-invite
FIREFLIES_NOTETAKER_EMAIL	The Fireflies bot's calendar-invite address
EMAIL_PROVIDER_API_KEY	Session confirmation emails (SendGrid/Postmark/SES — provider not yet chosen)
EMAIL_FROM_ADDRESS	Sender address for confirmation emails

Google Calendar (you confirmed you have these — never paste the actual values in chat, only set them directly in your hosting

platform's dashboard)

Variable	Account
GOOGLE_CALENDAR_INDIVIDUAL_CLIENT_ID / _CLIENT_SECRET / _REFRESH_TOKEN	petrazim.solutions@gmail.com
GOOGLE_CALENDAR_CORPORATE_CLIENT_ID / _CLIENT_SECRET / _REFRESH_TOKEN	contact.petrazim@gmail.com

Note: the actual OAuth call code for these isn't built yet — only the connector-status model/UI (ConnectorCards.tsx, ExternalConnector table). Setting these env vars is necessary but not sufficient; the Google Calendar API integration itself is still queued work.

One-time setup script (not a running env var — used once, then discard)

Variable	Used by
SUPER_ADMIN_EMAIL	seed_super_admin.py — Adekunle Oke's account
SUPER_ADMIN_INITIAL_PASSWORD	Same script — unset this from your environment immediately after running it once

Frontend

Variable	Used by
VITE_API_URL	Points the frontend at the deployed backend URL

Step 4 — Deploy (per the agreed plan)

Once Claude Code confirms the merged app runs locally without errors:

1. Backend to Render: web service + managed Postgres. Set every backend env var above in Render's dashboard. Run the SQL migrations against the Render Postgres instance.
2. Frontend to Vercel: point VITE_API_URL at the Render backend's URL. This resolves the earlier stalled Vercel connector attempt — worth retrying that connector now, or deploying fresh from the merged repo via Vercel's GitHub integration instead, which sidesteps the connector approval issue entirely since it doesn't go through this chat's tool-approval flow.
3. Verify: register a test account, confirm a checkout session can be created (even if it 503s on the payment provider stub, confirm the route responds correctly), confirm Telegram webhook receives a test join request, confirm the facilitator calendar loads real data from Postgres instead of erroring.

That last verification pass is really the first moment this entire build gets tested end-to-end as one system — everything up to now has been individually-tested components. That's expected and fine; it's exactly why "merge first, then one real deploy" was the right call over deploying pieces separately along the way.